Deployment module

- APIs
  - History of APIs
  - FASTAPI
  - GRCP
  - Web-sockets, SSE
  - A2A

- Dockers and Containers

Please install docker Desktop and account of docker hub

Github

Make a free account on Google cloud

Claim 200$ credits

1. Docker hub - Create account
2. Docker desktop - Install software and login
3. cloud.console.google.com - Get a free trial activated (Add Debit/Credit card) - Purpose - Learning
4. upstash

5. AWS
6. Github

Next week

Upcoming sessions -

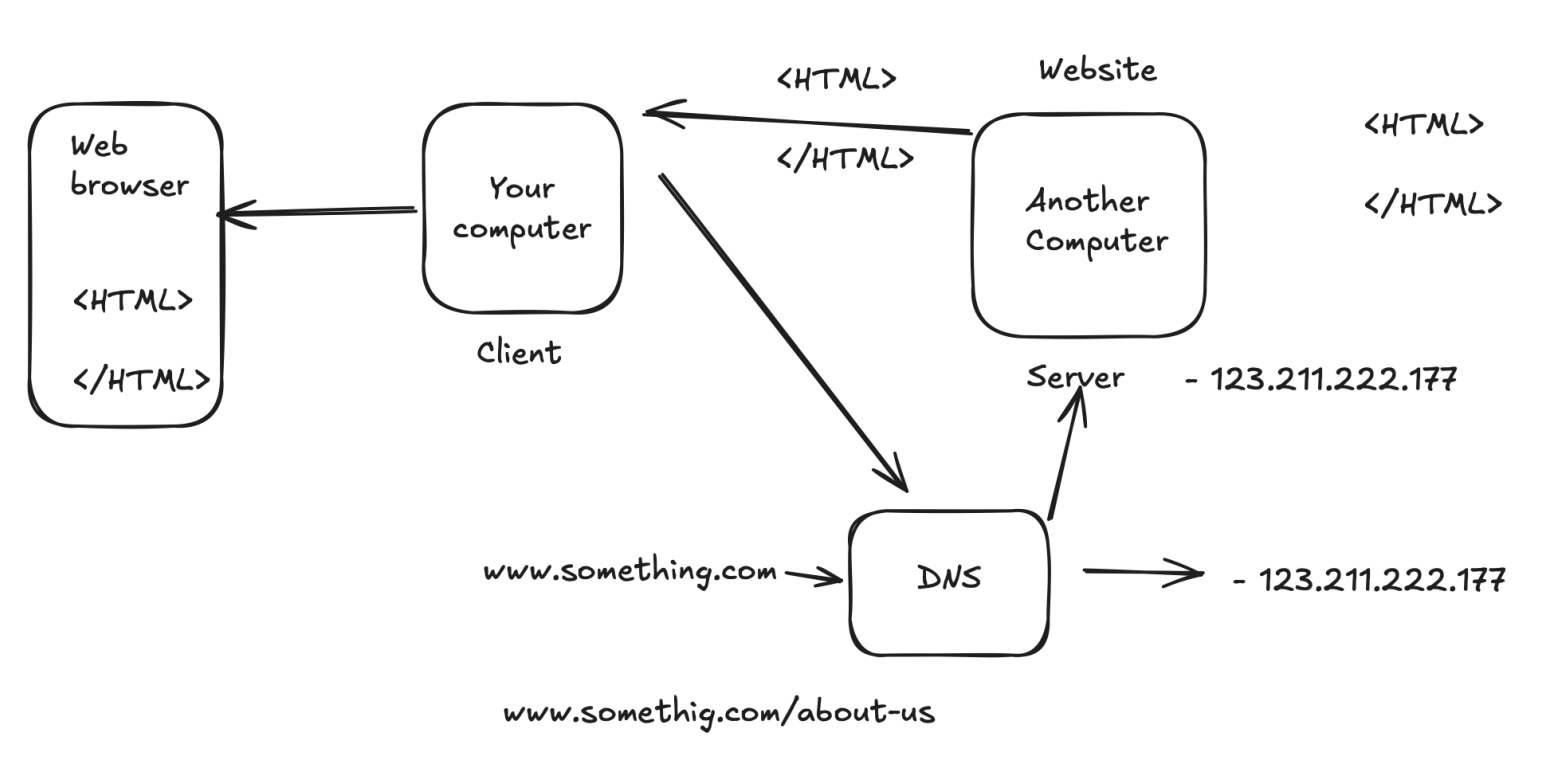
- Dockers and containers
- Github for code maintenance and versioning
- Google cloud for deploying
- CICD

More topics:

- Production memory
- A2A protocols
- Monitoring - Custom metrics, Custom dashboards
- Evaluation - Offline evaluation, online evaluation, LLM as a judge

APIs

It all started with websites


Web browser
<HTML>
</HTML>
Your computer
Client
<HTML>
</HTML>
Website
Another Computer
Server
- 123.211.222.177
www.something.com
DNS
- 123.211.222.177

<HTML>

</HTML>

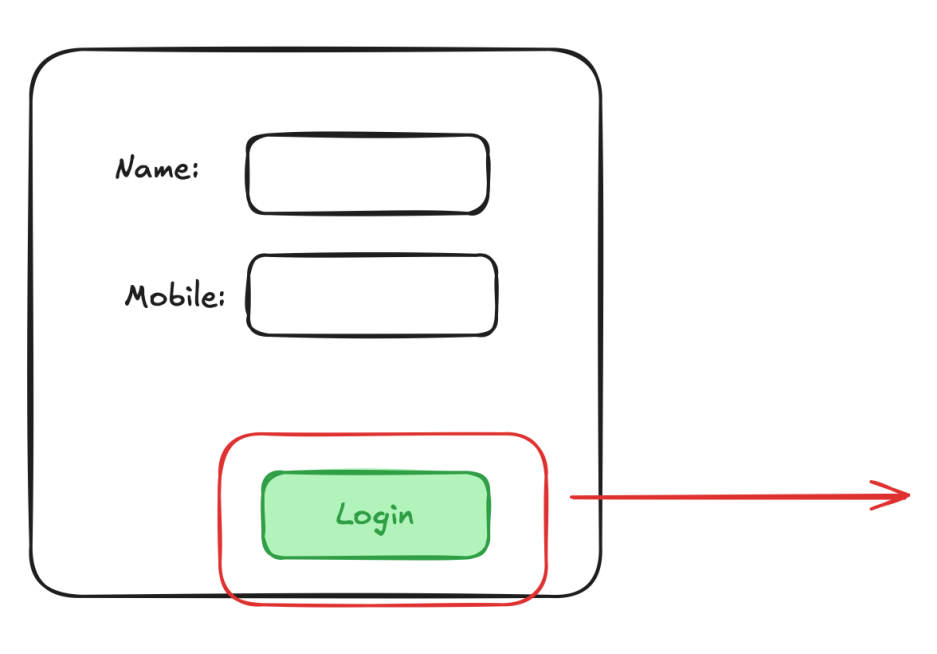
For interaction
- Format
- Protocol

www.somethig.com/about-us

Protocol - HTTP → How to behave

www.somethig.com/about-us → GET

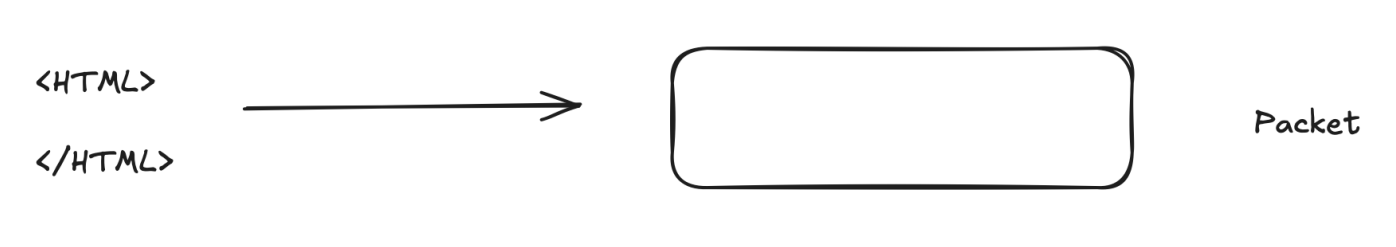
GET /about-us HTTP/1.1
Host: www.something.com
Accept: text/html


Name:
Mobile:
Login

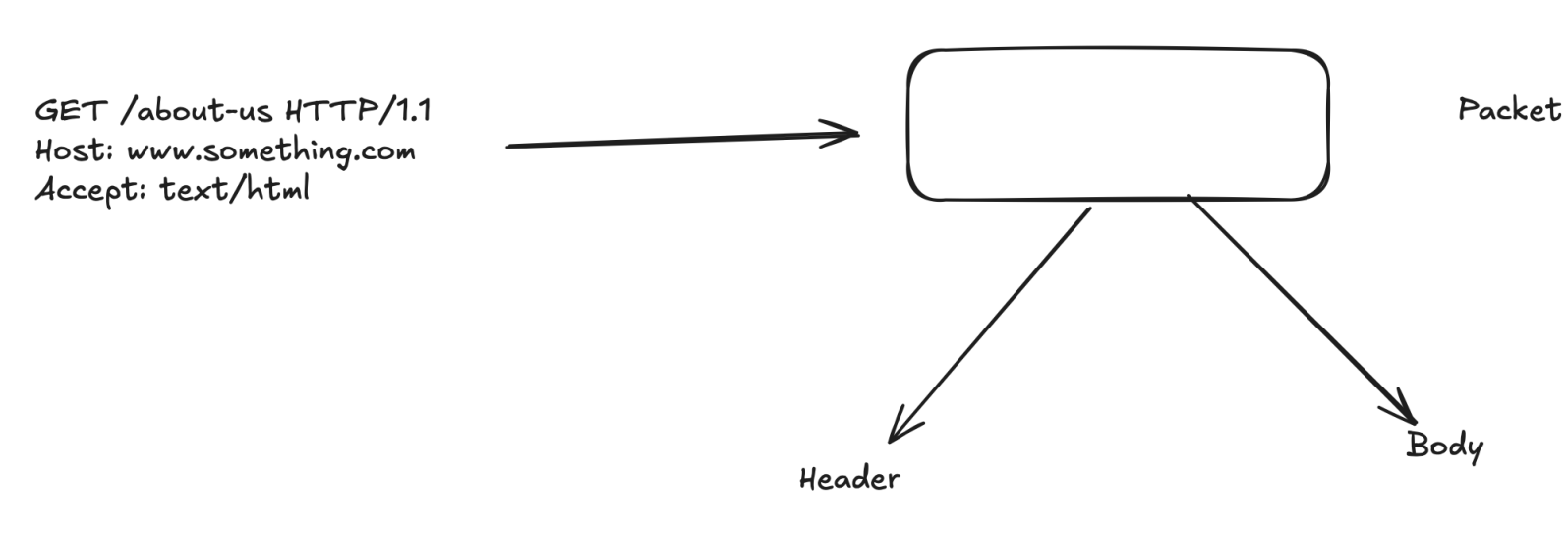
You're not immediately asking for data, first you're sending some data

POST

HTTP defines rules on how data will be sent over internet -

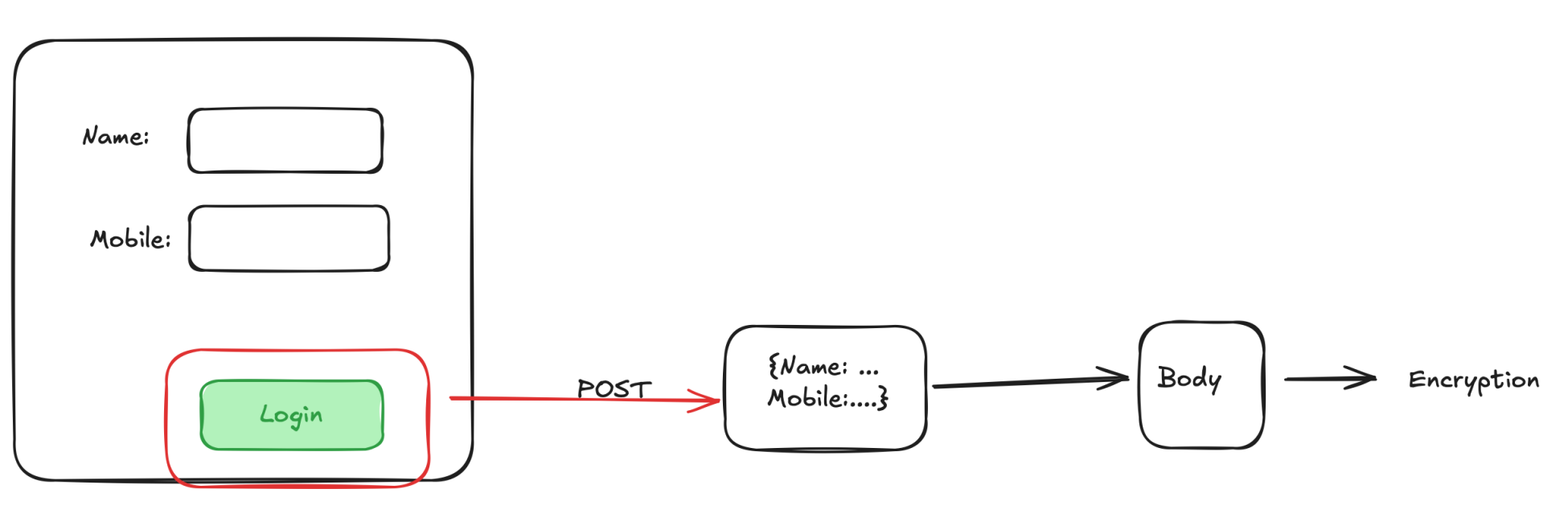


Packet


GET /about-us HTTP/1.1
Host: www.something.com
Accept: text/html
Header
Body

Packet

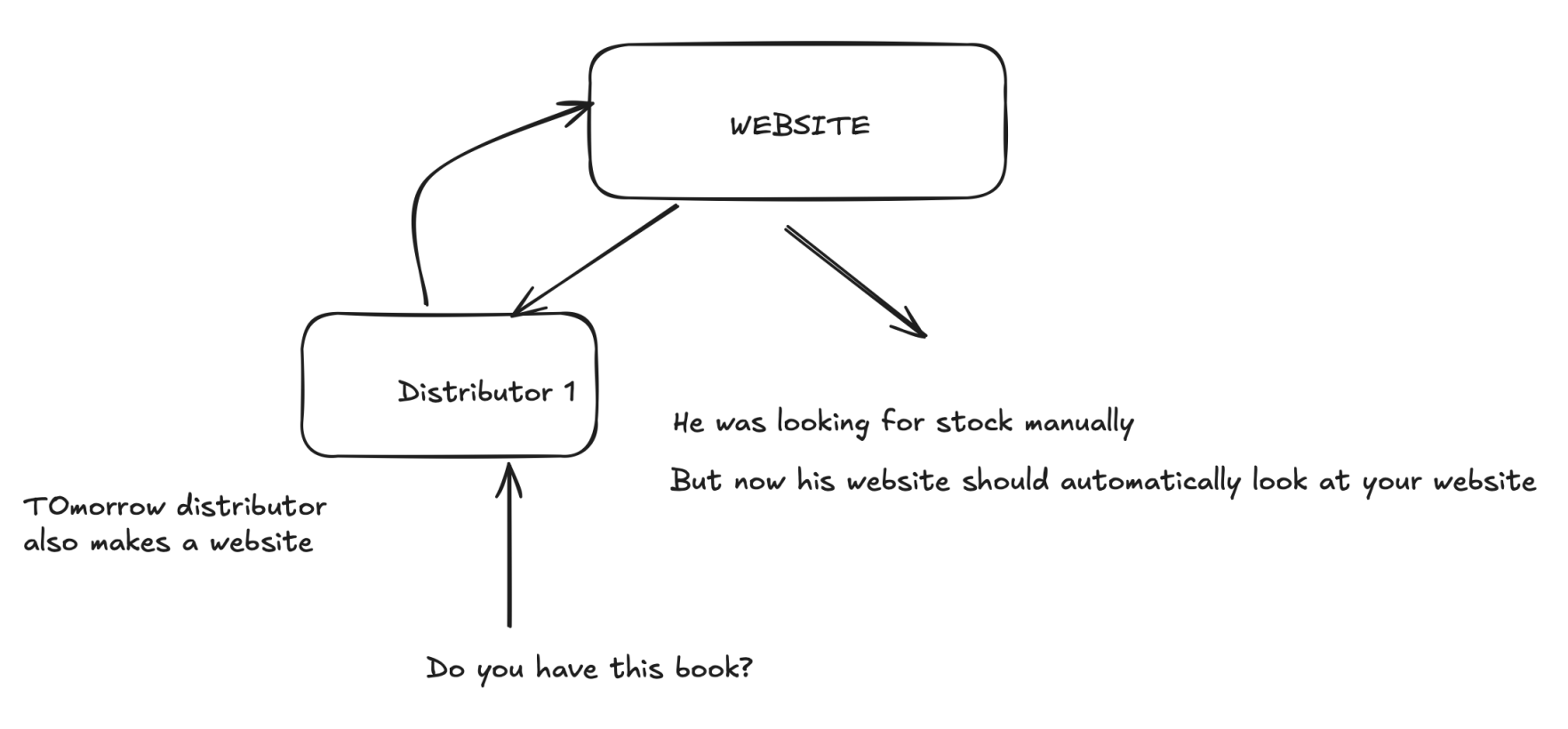
GET /about-us HTTP/1.1
Host: www.something.com
Accept: text/html → Header → Available in the link


Name:
Mobile:
Login
POST
{Name: ...
Mobile:....}
Body
Encryption

1. Methods to send and receive data
2. Security via encryption

The need to communicate

Let's say we have a book store website -


WEBSITE
Distributor 1
TOmorrow distributor
also makes a website
Do you have this book?

He was looking for stock manually

But now his website should automatically look at your website

```
1.

Request:
GET /books/101.html

Response:
<h1>Designing Data-Intensive Applications</h1>
<p class="price">₹799</p>


price = page.find(class_="price").text
```

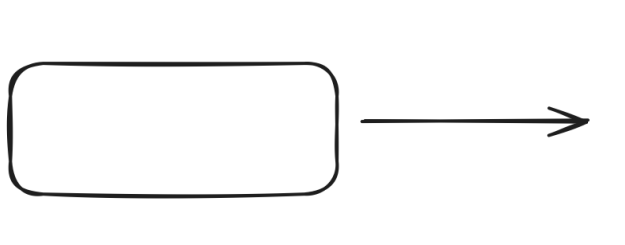
```
This is Scraping

Tomorrow the website changes

<span class="selling-price">₹799</span>
```

A stable separation between:

Presentation for humans
Data for software

1. CGI

- COMMON GATEWAY INTERFACE

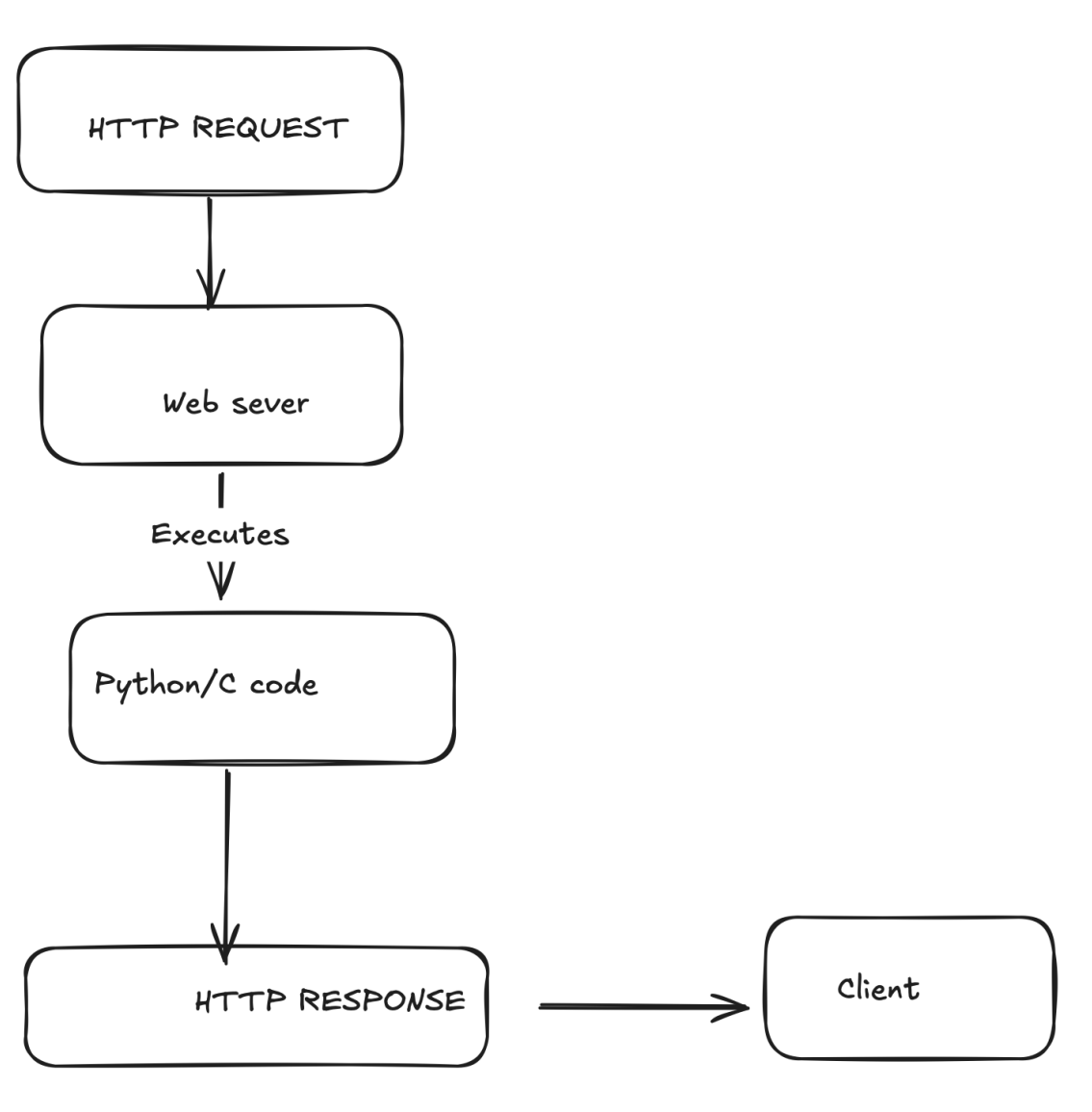


def get_book(id)

www.bookstore.com/get_book

GET /cgi-bin/book?action=get_book&id=101

Response:
101|Designing Data-Intensive Applications|799|12


HTTP REQUEST
Web sever
Executes
Python/C code
HTTP RESPONSE
Client

Each company invents its own format:

101|DDIA|799|12

Another returns:

id=101;title=DDIA;price=799

Another returns:

BOOK_ID:101
BOOK_PRICE:799

There was no standard on how the data should be sent back

There is no common definition for:
- calling an operation
- representing data types
- returning results
- representing errors

2. XML-RPC

www.bookstore.com

RPC means Remote Procedure Call.

The client writes:

book = server.getBook(101)

getBook(101)
↓ serialize
XML request
↓ HTTP
Remote server
↓ execute
getBook
↓ serialize
XML response

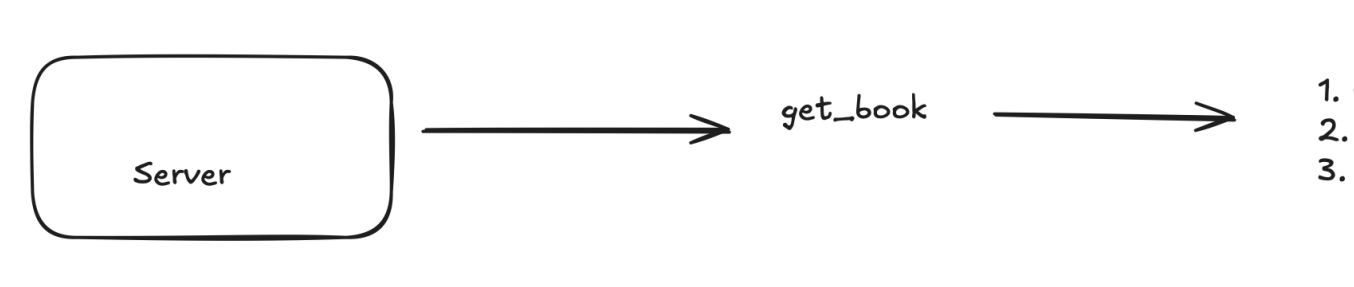
{"book_id": 101}
↓

```
<struct>
  <member>
    <name>book_id</name>
    <value><int>101</int></value>
  </member>
</struct>
```

```
<methodCall>
  <methodName>getBook</methodName>
  <params>
    <param>
      <value><int>101</int></value>
    </param>
  </params>
</methodCall>
```

XML-RPC standardizes:
- method name
- parameters
- basic data types
- response structure
- fault structure

Large companies need more than a standardized function call.


Server
get_book

1. what other services do you provide
2. What exact fields are required.
3. What is the data type of each field.

3. SOAP

```
<soap:Envelope
 xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">

 <soap:Header>
  <Authentication>
   <ApiKey>partner-123</ApiKey>
  </Authentication>
 </soap:Header>

 <soap:Body>
  <GetBookRequest>
   <BookId>101</BookId>
  </GetBookRequest>
 </soap:Body>
</soap:Envelope>
```

SOAP is a standardized XML messaging protocol.

Part	Purpose
Envelope	Marks the document as a SOAP message
Header	Carries metadata such as authentication
Body	Contains the business operation and data
Fault	Represents an error

WSDL

Service: BookService
Address: https://bookstore.com/soap

Operation: GetBook
Input: BookId as integer
Output: Book
Possible fault: BookNotFound

client = BookServiceClient(wsdl_url)
book = client.GetBook(BookId=101) → Still doing a RPC

XSD

XML Schema Definition.

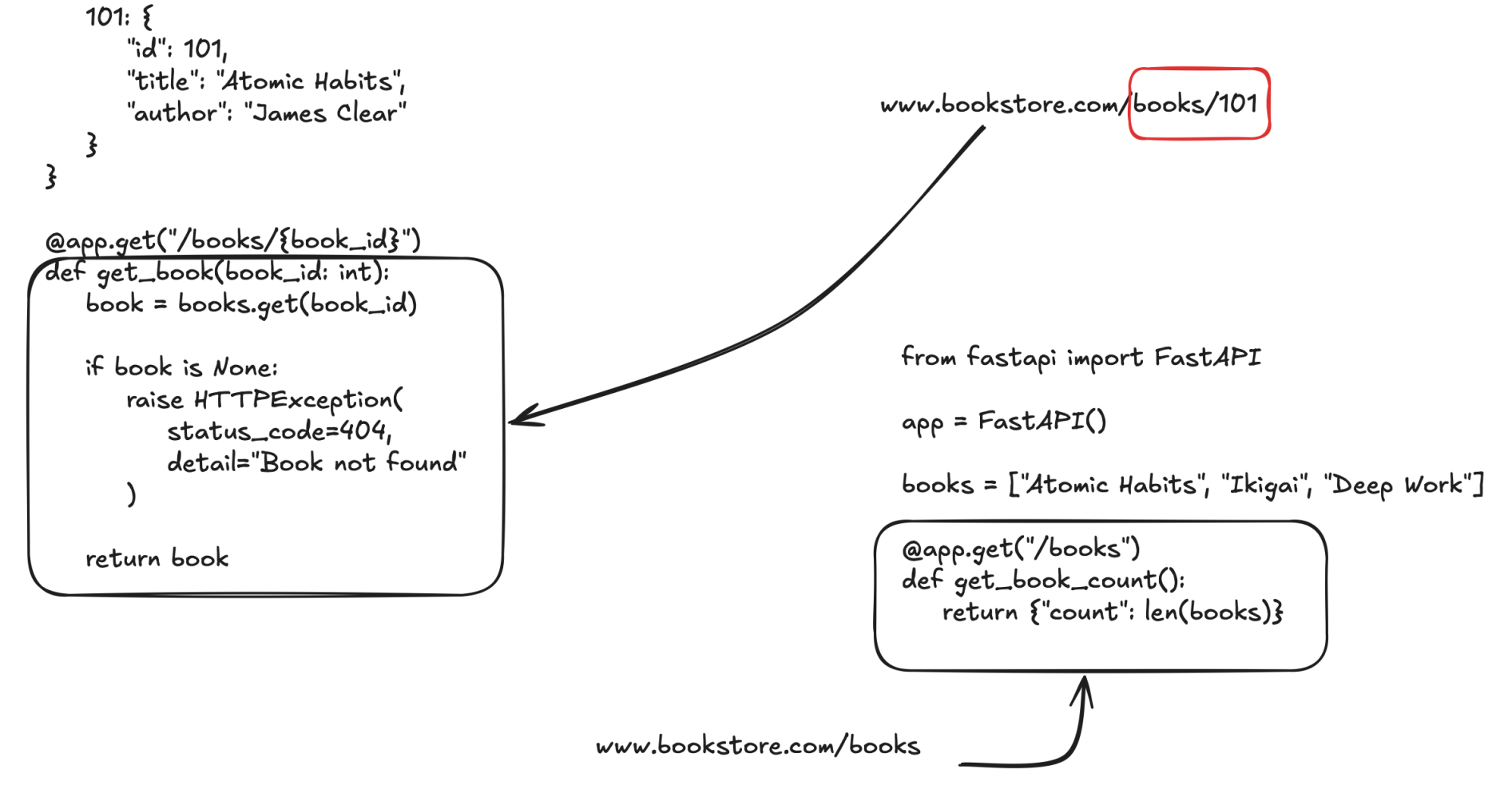
```
<xs:element
  name="BookId"
  type="xs:integer"/>
```

client = BookServiceClient(wsdl_url)
book = client.GetBook(BookId=101) → <BookId>101</BookId>

client = BookServiceClient(wsdl_url)
book = client.GetBook(BookId="Banana") → <BookId>Banana</BookId> ✗

A developer retrieving one book must deal with:
- SOAP envelopes
- XML namespaces
- WSDL
- XSD
- generated clients
- large XML messages
- complicated tooling

4. REST: represent resources through HTTP



```
from fastapi import FastAPI, HTTPException

app = FastAPI()

books = {
    101: {
        "id": 101,
        "title": "Atomic Habits",
        "author": "James Clear"
    }
}

@app.get("/books/{book_id}")
def get_book(book_id: int):
    book = books.get(book_id)

    if book is None:
        raise HTTPException(
            status_code=404,
            detail="Book not found"
        )

    return book
```

www.bookstore.com/books/101

```
from fastapi import FastAPI

app = FastAPI()

books = ["Atomic Habits", "Ikigai", "Deep Work"]

@app.get("/books")
def get_book_count():
    return {"count": len(books)}
```

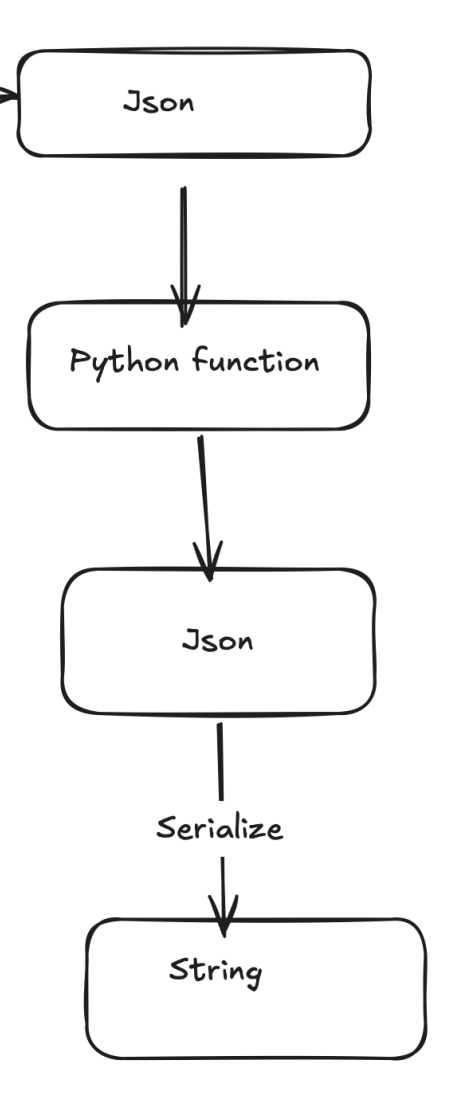
www.bookstore.com/books

When someone opens - www.bookstore.com/books → GET /books →

```
{
  "count": 3
}
```

Why FastAPI

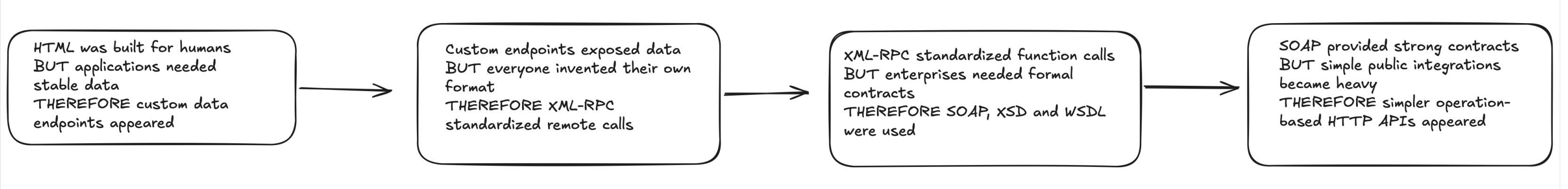
www.bookstore.com/books/101 → GET → /books/book_id=101 →


Json
Python function
Json
Serialize
String

Converted to Bytes and sent over internet as packets

Uvicorn — Server

It gives your website/service an environment on the host computer and exposes the functions as apis



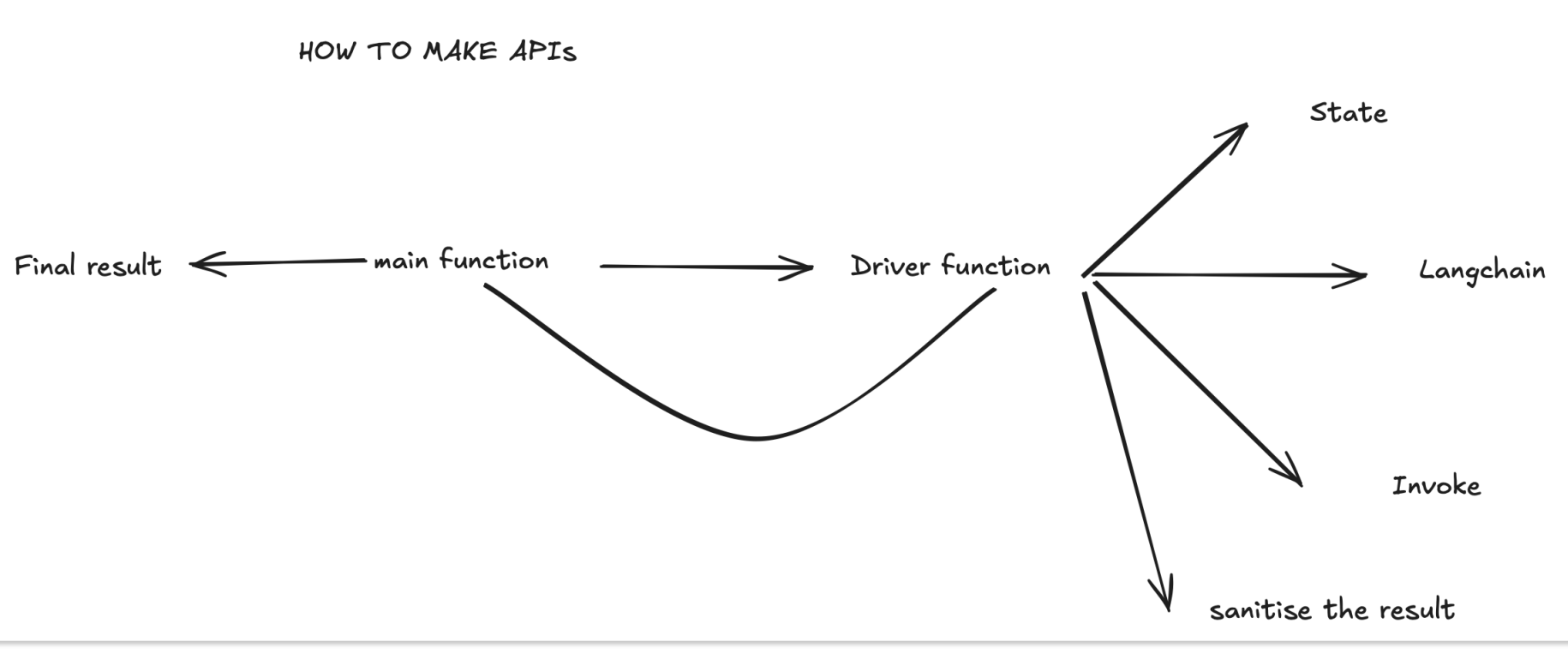
HTML was built for humans
BUT applications needed stable data
THEREFORE custom data endpoints appeared

Custom endpoints exposed data
BUT everyone invented their own format
THEREFORE XML-RPC standardized remote calls

XML-RPC standardized function calls
BUT enterprises needed formal contracts
THEREFORE SOAP, XSD and WSDL were used

SOAP provided strong contracts
BUT simple public integrations became heavy
THEREFORE simpler operation-based HTTP APIs appeared

HOW TO MAKE APIs


Final result
main function
Driver function
State
Langchain
Invoke
sanitise the result